Firebase + Flutter

A 1-Hour Practical Training

Auth · Cloud Firestore · live data · a clean app pattern you can reuse today

Today's agenda

Time	Topic
0:00 – 0:05	What Firebase gives you, and why it pairs well with Flutter
0:05 – 0:15	Project setup — Firebase console, FlutterFire CLI, packages
0:15 – 0:25	Firebase Authentication — email/password sign-up & sign-in
0:25 – 0:40	Cloud Firestore — collections, documents, CRUD, live streams
0:40 – 0:50	A clean pattern: Model → Repository → State management
0:50 – 0:55	Common mistakes and how to avoid them
0:55 – 1:00	Cheat sheet & wrap-up

How to use this handout: it's written so you can follow along on your laptop and re-run every snippet later. Code blocks are complete enough to paste in with only your own project's names swapped in — nothing here is pseudocode.

1 What Firebase gives you

0:00–0:05

Firebase is a set of backend services from Google that your Flutter app talks to directly — no server of your own to write, host, or patch at 2am.

The four pieces most apps start with:

- **Authentication** — sign-up/sign-in (email+password, phone, Google, etc.) without building your own login system.
- **Cloud Firestore** — a NoSQL database that *syncs live*: every device watching the same data updates instantly when it changes.
- **Cloud Storage** — file/image uploads (profile photos, attachments).
- **Cloud Functions** — small pieces of server-side code, for things a phone shouldn't do directly (sending emails, charging a card).

> **NOTE** — For most CRUD-style apps (shop management, to-do lists, chat, booking apps) Auth + Firestore alone gets you to a working product — that's what this session focuses on.

Why it fits Flutter well

Firebase publishes official Flutter packages (the `Firebase***` family) that plug straight into Dart's async/Stream model — so a live Firestore query is just a `Stream` you can feed into a `StreamBuilder` or a Bloc, and the UI updates itself.

2 Project setup

0:05–0:15

Step 1 — Create the Firebase project

- Go to `console.firebase.google.com` → **Add project**.
- Name it, skip Google Analytics for a training/demo project (you can add it later).
- Once created, add an app: choose Android/iOS/Web depending on your target — Flutter can target all three from one codebase.

Step 2 — Wire it into your Flutter project (FlutterFire CLI)

The FlutterFire CLI generates the platform config files for you — far less error-prone than copying them by hand.

```
# one-time install
dart pub global activate flutterfire_cli

# from your Flutter project's root folder
flutterfire configure
# -> pick your Firebase project, then pick the platforms to generate for
# -> creates lib/firebase_options.dart automatically
```

Step 3 — Add the packages

```
flutter pub add firebase_core firebase_auth cloud_firestore
```

This adds current versions to `pubspec.yaml` automatically — no need to type version numbers by hand.

Step 4 — Initialize before the app runs

```
import 'package:firebase_core/firebase_core.dart';
import 'firebase_options.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  runApp(const MyApp());
}
```

WATCH OUT — Forgetting `WidgetsFlutterBinding.ensureInitialized()` before an `await` in `main()` is the #1 reason "Firebase not initialized" crashes happen on a fresh project.

3 Firebase Authentication

0:15–0:25

Enable it first: Firebase console → **Build** → **Authentication** → **Sign-in method** → turn on **Email/Password** (or whichever provider you need).

Sign up a new user

```
Future<void> signUp(String email, String password) async {
  final credential = await FirebaseAuth.instance
      .createUserWithEmailAndPassword(email: email, password: password);
  print('Created: ${credential.user?.uid}');
}
```

Sign in an existing user

```
Future<void> signIn(String email, String password) async {
  try {
    await FirebaseAuth.instance
        .signInWithEmailAndPassword(email: email, password: password);
  } on FirebaseAuthException catch (e) {
    // e.code: 'user-not-found', 'wrong-password', 'invalid-email', ...
    print('Login failed: ${e.code}');
  }
}
```

Know who's logged in, live

```
// Fires immediately with the current user (or null), then again on
// every sign-in/sign-out — perfect for routing Splash -> Login/Home.
StreamBuilder<User?>(
  stream: FirebaseAuth.instance.authStateChanges(),
  builder: (context, snapshot) {
    final user = snapshot.data;
    return user == null ? const LoginScreen() : const HomeScreen();
  },
)
```

Sign out

```
await FirebaseAuth.instance.signOut();
```

WATCH OUT — Firebase Auth's email field must look like a real email address. A common trick for "username only" logins is to turn the typed username into a fixed placeholder email behind the scenes (e.g. `'$username@yourapp.app'`) — just be aware "forgot password" won't work for that account unless you also store a real email somewhere you look up first.

4 Cloud Firestore

0:25–0:40

The mental model

- A **collection** is like a folder (e.g. `products`).
- A **document** is a single record inside it (e.g. one product), identified by an ID.
- A document holds **fields** — a map of key/value pairs, similar to a JSON object.
- Collections are created automatically the first time you save a document into them — there's no separate "create table" step.

Write a document

```
final products = FirebaseFirestore.instance.collection('products');

// Auto-generated ID:
await products.add({
  'name': 'iPhone 14',
  'price': 68999,
  'stockQty': 12,
  'createdAt': FieldValue.serverTimestamp(),
});

// Or choose your own ID:
await products.doc('sku-14pro').set({'name': 'iPhone 14 Pro'});
```

Use `FieldValue.serverTimestamp()` instead of `DateTime.now()` — it's Firebase's clock, not the phone's, so it can't be wrong because someone's device time is off.

Read once

```
final snapshot = await products.doc('sku-14pro').get();
if (snapshot.exists) {
  final data = snapshot.data()!;
  print(data['name']);
}
```

Read live (the feature that matters most)

```
// Every device watching this fires again the instant the data changes
// anywhere -- another user's phone, the console, a Cloud Function.
Stream<QuerySnapshot> stream = products
    .where('shopId', isEqualTo: 'main')
    .snapshots();

StreamBuilder<QuerySnapshot>(
  stream: stream,
  builder: (context, snapshot) {
    if (!snapshot.hasData) return const CircularProgressIndicator();
    final docs = snapshot.data!.docs;
    return ListView(
      children: docs.map((d) => Text(d['name'] as String)).toList(),
    );
  },
)
```

Update and delete

```
// Update just some fields -- doesn't touch the rest of the document
await products.doc('sku-14pro').update({'stockQty': 11});

// Atomically increase/decrease a number (safe even with many writers at once)
await products.doc('sku-14pro').update({'stockQty': FieldValue.increment(-1)});

await products.doc('sku-14pro').delete();
```

Multiple writes that must all succeed together

Example: completing a sale should write the sale record *and* reduce stock — never just one of the two. A `runTransaction` makes that atomic.

```
await FirebaseFirestore.instance.runTransaction((txn) async {
  final ref = products.doc(productId);
  final snap = await txn.get(ref);          // all reads first
  final stock = (snap.data()?['stockQty'] as num).toInt();
  if (stock < qty) throw Exception('Not enough stock');

  txn.set(salesCollection.doc(), saleData);  // then all writes
  txn.update(ref, {'stockQty': FieldValue.increment(-qty)});
});
```

COMMON MISTAKE — Transactions require every `.get()` to happen before any `.set()`/`.update()` in the same transaction. Mixing the order throws at runtime.

5 A clean, reusable app pattern

0:40–0:50

Calling Firestore directly from your widgets works for a demo, but it gets messy fast. A small, three-layer pattern keeps things testable and easy to extend:

- **Model** — a plain Dart class for one record, with `fromMap`/`toMap` to convert to and from Firestore's map format.
- **Repository** — the only place that talks to Firestore directly. Exposes clean methods like `watchProducts()` and `addProduct()`.
- **UI / state management** (Bloc, Provider, Riverpod — your choice) — asks the repository for data and streams, never imports `cloud_firestore` itself.

Model

```
class Product {
  final String id, name;
  final double price;
  final int stockQty;

  Product({required this.id, required this.name,
           required this.price, required this.stockQty});

  factory Product.fromMap(String id, Map<String, dynamic> map) => Product(
    id: id,
    name: map['name'] as String,
    price: (map['price'] as num).toDouble(),
    stockQty: (map['stockQty'] as num).toInt(),
  );

  Map<String, dynamic> toMap() => {
    'name': name, 'price': price, 'stockQty': stockQty,
  };
}
```

Repository

```
class ProductRepository {
  final _col = FirebaseFirestore.instance.collection('products');

  Stream<List<Product>> watchProducts() {
    return _col.snapshots().map((snap) => snap.docs
        .map((d) => Product.fromMap(d.id, d.data())).toList());
  }

  Future<void> addProduct(Product p) => _col.add(p.toMap());
}
```

Why bother with this split?

- If you ever swap Firestore for another backend, only the repository changes.
- Widgets stay simple — they just render whatever the repository/Bloc gives them.
- Much easier to unit-test: fake the repository, no real Firestore needed for widget tests.

6 Common mistakes

0:50–0:55

Mistake	Fix
Firestore security rules left wide open (test mode) past day 30	Test mode auto-expires and locks everyone out. Write real rules before you ship, even simple ones.
Reading a field that might not exist without a null check	Use `map['field'] as Type?` or provide a fallback with `??`.
Sorting a live query server-side by a field with no index yet	Firestore asks for a composite index the first time — click the console link it gives you, or sort client-side in Dart to avoid the index entirely.
Using `DateTime.now()` for timestamps	Use `FieldValue.serverTimestamp()` — consistent across every device.
Two related writes (e.g. sale + stock) done as separate calls	Wrap them in `runTransaction` so they succeed or fail together.
Calling `Firebase.initializeApp()` without awaiting it first	Always `await` it in `main()` before `runApp()`.

7 Cheat sheet

0:55–1:00

Task	Code
Init	`await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform);`
Sign up	`FirebaseAuth.instance.createUserWithEmailAndPassword(email:, password:)`
Sign in	`FirebaseAuth.instance.signInWithEmailAndPassword(email:, password:)`
Sign out	`FirebaseAuth.instance.signOut()`
Who's logged in (live)	`FirebaseAuth.instance.authStateChanges()`
Collection ref	`FirebaseFirestore.instance.collection('name')`
Add doc (auto id)	`collection.add({...})`
Set doc (chosen id)	`collection.doc('id').set({...})`
Update fields	`collection.doc('id').update({...})`
Delete doc	`collection.doc('id').delete()`
Read once	`collection.doc('id').get()`
Read live (query)	`collection.where(...).snapshots()`
Atomic +/- a number	`FieldValue.increment(n)`
Server-side timestamp	`FieldValue.serverTimestamp()`
Multiple linked writes	`FirebaseFirestore.instance.runTransaction((txn) async {...})`

Where to go next

- Firestore security rules — lock down who can read/write what before going live.
- Cloud Storage — for photo/file uploads.
- Cloud Functions — server-side logic (emails, payment webhooks, scheduled jobs).
- Official docs: `firebase.google.com/docs/flutter/setup`

End of handout — thanks for following along. Keep this as a reference for your next Firebase + Flutter feature.